

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Code Challenge (High)

Assessment Tool Weightage: 35%

Dr

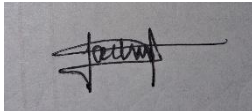
QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	5
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	5
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	5
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	5
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	5
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	5
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	5
8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	5

9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	5
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	5

Code of Conduct:

I J a s h w a n t h . G . R (1 9 2 5 2 4 3 8 4) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results

Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including	Handles most test cases	Misses important edge	Fails on multiple test
		edge and hidden cases		cases	cases

Q1. Heap File Organization (Insert, Delete, Sequential Scan)

Bloom's Level: BL3 - Apply | Marks: 5

Approach

Records are appended to a list in arrival order (the defining trait of a heap/unordered file). Delete is done via tombstone marking so surviving records never need to shift; the freed slot is tracked and reused by a later insert, exactly as real heap files reuse deleted space. Sequential scan simply walks every slot in physical order, skipping tombstones.

Code (Python)

```
"""
```

Q1: Heap File Organization

Supports insert, delete, and sequential scan operations.

Records are stored in the order they arrive (unordered).

Delete is implemented via tombstone marking (common heap-file technique)

so that record slots do not need to be shifted.

```
"""
```

```
class HeapFile:

    def __init__(self):
        self.records = []          # list of dicts: {'id':..., 'name':..., 'deleted':
bool}

        self.free_slots = []      # reusable slots left behind by deletions

    def insert(self, record_id, data):
        """Insert a new record. Reuse a deleted slot if available (heap
behaviour)."""

        record = {"id": record_id, "data": data, "deleted": False}

        if self.free_slots:
            slot = self.free_slots.pop()

            self.records[slot] = record

            print(f"Inserted record {record_id} into reused slot {slot}")

        else:
            self.records.append(record)
```

```

        print(f"Inserted record {record_id} into new slot {len(self.records) -
1}")

def delete(self, record_id):
    """Delete by marking the slot as a tombstone (no shifting needed)."""
    for i, rec in enumerate(self.records):
        if rec and not rec["deleted"] and rec["id"] == record_id:
            rec["deleted"] = True
            self.free_slots.append(i)
            print(f"Deleted record {record_id} (slot {i} marked free)")
            return True

    print(f"Record {record_id} not found")

    return False

def sequential_scan(self):
    """Scan every slot in physical order and print live records."""
    print("Sequential Scan:")

    found = False

    for i, rec in enumerate(self.records):
        if rec and not rec["deleted"]:
            print(f"  slot {i}: id={rec['id']}, data={rec['data']}")
            found = True

    if not found:
        print("  (heap file is empty)")

if __name__ == "__main__":
    heap = HeapFile()

    heap.insert(1, "Alice")
    heap.insert(2, "Bob")
    heap.insert(3, "Charlie")

    heap.sequential_scan()

    heap.delete(2)

    heap.sequential_scan()

    heap.insert(4, "David")    # reuses slot freed by delete(2)

```

```
heap.sequential_scan()
```

Sample Output

=== q1_heap_file.py ===

Inserted record 1 into new slot 0

Inserted record 2 into new slot 1

Inserted record 3 into new slot 2

Sequential Scan:

slot 0: id=1, data=Alice

slot 1: id=2, data=Bob

slot 2: id=3, data=Charlie

Deleted record 2 (slot 1 marked free)

Sequential Scan:

slot 0: id=1, data=Alice

slot 2: id=3, data=Charlie

Inserted record 4 into reused slot 1

Sequential Scan:

slot 0: id=1, data=Alice

slot 1: id=4, data=David

slot 2: id=3, data=Charlie

Q2. Static Hashing for a Student Database (500 records, chaining)

Bloom's Level: BL3 - Apply | Marks: 5

Approach

A fixed-size array of 97 buckets (a prime, for even distribution) is used. The hash function is $\text{student_id} \bmod 97$.

Collisions -- multiple ids mapping to the same bucket -- are resolved with chaining: each bucket is a Python list acting as a linked list of (id, name) pairs. The number of buckets never changes at runtime, which is what makes this 'static' hashing as opposed to dynamic/extendible hashing.

Code (Python)

"""

Q2: Static Hashing for a Student Database (500 records)

Hash table has a fixed number of buckets; collisions are handled using chaining (each bucket is a Python list acting as a linked list).

"""

```
NUM_BUCKETS = 97    # a prime number close to 500/5, gives good spread for chaining

class StaticHashTable:

    def __init__(self, num_buckets=NUM_BUCKETS):

        self.num_buckets = num_buckets

        self.buckets = [[] for _ in range(num_buckets)]

    def _hash(self, student_id):

        return student_id % self.num_buckets

    def insert(self, student_id, name):

        idx = self._hash(student_id)

        # chaining: append to the list at that bucket, even if it already has entries

        self.buckets[idx].append((student_id, name))

    def search(self, student_id):

        idx = self._hash(student_id)

        for sid, name in self.buckets[idx]:

            if sid == student_id:

                return name

        return None

    def delete(self, student_id):

        idx = self._hash(student_id)

        bucket = self.buckets[idx]

        for i, (sid, _) in enumerate(bucket):

            if sid == student_id:

                bucket.pop(i)

                return True

        return False
```

```

def bucket_load_report(self):
    max_chain = max(len(b) for b in self.buckets)
    used = sum(1 for b in self.buckets if b)
    print(f"Buckets used: {used}/{self.num_buckets}, longest chain: {max_chain}")

if __name__ == "__main__":
    table = StaticHashTable()
    # Simulate inserting 500 student records (student_id 1000..1499)
    for i in range(500):
        student_id = 1000 + i
        table.insert(student_id, f"Student_{i}")
    table.bucket_load_report()
    # Demonstrate search
    print("Search 1250:", table.search(1250))
    print("Search 9999:", table.search(9999))
    # Demonstrate collision handling explicitly
    print("\nExample bucket contents (bucket 0), showing chaining:")
    idx = 0
    print(table.buckets[idx][:5], "..." if len(table.buckets[idx]) > 5 else "")
    # Demonstrate delete

```



```

table.delete(1250)

print("After delete, search 1250:", table.search(1250))

```

Sample Output

=== q2_static_hashing.py ===

Buckets used: 97/97, longest chain: 6

Search 1250: Student_250

Search 9999: None

Example bucket contents (bucket 0), showing chaining:

```

[(1067, 'Student_67'), (1164, 'Student_164'), (1261, 'Student_261'), (1358, 'Student_358'), (1455,
'Student_455')]

```

After delete, search 1250: None

Q3. Sorted File Organization + Binary Search by Primary Key

Bloom's Level: BL4 - Analyze | Marks: 5

Approach

Every insert uses bisect to place the new record at its correct sorted position, so the file is always physically ordered by primary key (the defining property of a sorted file). Retrieval by key then uses classic binary search ($O(\log n)$) instead of a linear scan, and the program reports the number of comparisons used to make the efficiency gain explicit.

Code (Python)

```

"""

```

Q3: Sorted File Organization + Binary Search retrieval by primary key.

Records are always kept sorted by primary key (id) after insertion,

which is the defining property of a sorted (ordered) file.

```

"""

```

```

class SortedFile:

    def __init__(self):

        self.records = []    # list of (id, data), always kept sorted by id

    def insert(self, record_id, data):

        # find correct sorted position and insert (bisect keeps file ordered)

```

```

import bisect

keys = [r[0] for r in self.records]

pos = bisect.bisect_left(keys, record_id)

self.records.insert(pos, (record_id, data))

print(f"Inserted ({record_id}, {data}) -> file re-sorted, position {pos}")

def binary_search(self, key):
    """Standard binary search over the sorted record list. O(log n)."""
    lo, hi = 0, len(self.records) - 1

    comparisons = 0

    while lo <= hi:
        comparisons += 1

        mid = (lo + hi) // 2

        mid_key = self.records[mid][0]

        if mid_key == key:
            print(f"Found key {key} at index {mid} in {comparisons} comparisons")

            return self.records[mid]

        elif mid_key < key:
            lo = mid + 1

        else:
            hi = mid - 1

    print(f"Key {key} not found ({comparisons} comparisons)")

    return None

def display(self):
    print("Sorted file contents:", self.records)

if __name__ == "__main__":
    sf = SortedFile()

    for rid, name in [(50, "Eve"), (10, "Alice"), (30, "Bob"), (20, "Carol"), (40,
"Dan")]:
        sf.insert(rid, name)

    sf.display()

    sf.binary_search(30)

```

```
sf.binary_search(25)
```

Sample Output

=== **q3_sorted_file.py** ===

Inserted (50, Eve) -> file re-sorted, position 0

Inserted (10, Alice) -> file re-sorted, position 0

Inserted (30, Bob) -> file re-sorted, position 1

Inserted (20, Carol) -> file re-sorted, position 1

Inserted (40, Dan) -> file re-sorted, position 3

Sorted file contents: [(10, 'Alice'), (20, 'Carol'), (30, 'Bob'), (40, 'Dan'), (50, 'Eve')]

Found key 30 at index 2 in 1 comparisons

Key 25 not found (3 comparisons)

Q4. B-Tree of Order 4 (Insert, Search, Display)

Bloom's Level: BL3 - Apply | Marks: 5

Approach

Order 4 means each node holds at most 3 keys and 4 children. Insertion uses the standard 'split full child on the way down' technique: before descending into a full child, it is split and its middle key is pushed up to the parent, guaranteeing the root is never over capacity when the recursive insert reaches a leaf. The keys 10, 20, 5, 6, 12, 30, 7, 17 are inserted in the exact order given in the question, and the resulting tree is shown level by level.

Code (Python)

```
"""
```

Q4: B-Tree of order 4 (max 3 keys per node, max 4 children).

Supports insert, search, and display (level-order).

Insert sequence: 10, 20, 5, 6, 12, 30, 7, 17

```
"""
```

```
ORDER = 4                                # order m = 4

MAX_KEYS = ORDER - 1                      # 3

class BTreeNode:

    def __init__(self, leaf=True):

        self.keys = []

        self.children = []

        self.leaf = leaf

class BTree:

    def __init__(self, order=ORDER):

        self.order = order

        self.max_keys = order - 1

        self.root = BTreeNode(leaf=True)

    def search(self, key, node=None):

        node = node or self.root

        i = 0

        while i < len(node.keys) and key > node.keys[i]:

            i += 1
```

```

        if i < len(node.keys) and node.keys[i] == key:
            return True

        if node.leaf:
            return False

        return self.search(key, node.children[i])

def insert(self, key):
    root = self.root

    if len(root.keys) == self.max_keys:
        new_root = BTreeNode(leaf=False)
        new_root.children.append(root)
        self._split_child(new_root, 0)
        self.root = new_root
        self._insert_non_full(new_root, key)
    else:
        self._insert_non_full(root, key)

def _insert_non_full(self, node, key):
    i = len(node.keys) - 1

    if node.leaf:
        node.keys.append(None)

        while i >= 0 and key < node.keys[i]:
            node.keys[i + 1] = node.keys[i]
            i -= 1

        node.keys[i + 1] = key
    else:
        while i >= 0 and key < node.keys[i]:
            i -= 1

        i += 1

        if len(node.children[i].keys) == self.max_keys:
            self._split_child(node, i)

            if key > node.keys[i]:
                i += 1

```

```

        self._insert_non_full(node.children[i], key)

def _split_child(self, parent, i):

order = self.order

    mid = (order - 1) // 2          # for order 4 -> mid index 1

    child = parent.children[i]

    new_node = BTreeNode(leaf=child.leaf)

    mid_key = child.keys[mid]

    new_node.keys = child.keys[mid + 1:]

    child.keys = child.keys[:mid]

    if not child.leaf:

        new_node.children = child.children[mid + 1:]

        child.children = child.children[:mid + 1]

    parent.children.insert(i + 1, new_node)

    parent.keys.insert(i, mid_key)

def display(self):

    """Level-order display of the tree."""

    print("B-Tree (level order):")

    level = [self.root]

    depth = 0

    while level:

        line = []

        next_level = []

        for node in level:

            line.append(str(node.keys))

            next_level.extend(node.children)

        print(f"  Level {depth}: " + " ".join(line))

        level = next_level

        depth += 1

if __name__ == "__main__":

    bt = BTree(order=4)

```

```
keys_to_insert = [10, 20, 5, 6, 12, 30, 7, 17]

for k in keys_to_insert:

    bt.insert(k)

    print(f"Inserted {k}")

bt.display()

for k in [12, 99]:

    print(f"Search {k}: {'Found' if bt.search(k) else 'Not found'}")
```

Sample Output

=== q4_btree.py ===

Inserted 10

Inserted 20

Inserted 5

Inserted 6

Inserted 12

Inserted 30

Inserted 7

Inserted 17

B-Tree (level order):

Level 0: [10, 20]

Level 1: [5, 6, 7] [12, 17] [30]

Search 12: Found

Search 99: Not found

Q5. B+ Tree with Leaf-Level Linking for Range Queries

Bloom's Level: BL3 - Apply | Marks: 5

Approach

Unlike a plain B-Tree, all data values live only in the leaf nodes; internal nodes store only routing keys. Every leaf keeps a 'next' pointer to the leaf immediately to its right, forming a singly linked list across all leaves in sorted key order. A range query therefore needs only ONE descent from the root to locate the starting leaf, after which it simply follows 'next' pointers -- this is the classic efficiency argument for B+ Trees over B-Trees in range scans.

Code (Python)

```
"""
```

Q5: B+ Tree with leaf-level linking for efficient range queries.

All actual data lives in the leaf nodes; leaves are linked left-to-right via a 'next' pointer so a range query can just walk the leaf chain after one descent from the root.

```
"""
```

```
ORDER = 4                # max children per internal node
MAX_LEAF_KEYS = ORDER - 1

class Node:
    def __init__(self, leaf=True):
        self.leaf = leaf
        self.keys = []
        self.children = []  # internal: child Nodes | leaf: values (parallel to
                             keys)
        self.next = None    # leaf-level link (only used when leaf=True)

class BPlusTree:
    def __init__(self, order=ORDER):
        self.order = order
        self.max_keys = order - 1
        self.root = Node(leaf=True)

    def search(self, key):
        node = self._find_leaf(key)
```



```

        for i, k in enumerate(node.keys):
            if k == key:
                return node.children[i]

        return None

def _find_leaf(self, key):
    node = self.root
    while not node.leaf:
        i = 0
        while i < len(node.keys) and key >= node.keys[i]:
            i += 1
        node = node.children[i]
    return node

def insert(self, key, value):
    leaf = self._find_leaf(key)
    self._insert_in_leaf(leaf, key, value)
    if len(leaf.keys) > self.max_keys:
        self._split_leaf(leaf)

def _insert_in_leaf(self, leaf, key, value):
    i = 0
    while i < len(leaf.keys) and leaf.keys[i] < key:
        i += 1
    leaf.keys.insert(i, key)
    leaf.children.insert(i, value)

def _split_leaf(self, leaf):
    mid = len(leaf.keys) // 2
    new_leaf = Node(leaf=True)
    new_leaf.keys = leaf.keys[mid:]
    new_leaf.children = leaf.children[mid:]
    leaf.keys = leaf.keys[:mid]
    leaf.children = leaf.children[:mid]
    # maintain the leaf-level linked list

```



```

        new_leaf.next = leaf.next

        leaf.next = new_leaf

        up_key = new_leaf.keys[0]

        self._insert_in_parent(leaf, up_key, new_leaf)

def _insert_in_parent(self, left, key, right):
    if left is self.root:
        new_root = Node(leaf=False)

        new_root.keys = [key]

        new_root.children = [left, right]

        self.root = new_root

        return

    parent = self._find_parent(self.root, left)

    i = parent.children.index(left)

    parent.keys.insert(i, key)

    parent.children.insert(i + 1, right)

    if len(parent.keys) > self.max_keys:
        self._split_internal(parent)

def _split_internal(self, node):
    mid = len(node.keys) // 2

    up_key = node.keys[mid]

    new_node = Node(leaf=False)

    new_node.keys = node.keys[mid + 1:]

    new_node.children = node.children[mid + 1:]

    node.keys = node.keys[:mid]

    node.children = node.children[:mid + 1]

    self._insert_in_parent(node, up_key, new_node)

def _find_parent(self, current, child):
    if current.leaf or all(not isinstance(c, Node) or c.leaf for c in
current.children):

        if child in current.children:

            return current

```

```

for c in current.children:
    if isinstance(c, Node) and not c.leaf:
        if child in c.children:
            return c

        result = self._find_parent(c, child)
        if result:
            return result

return None

def range_query(self, low, high):
    """Leaf-linking makes this efficient: descend once, then walk 'next'
    pointers."""

    result = []

    node = self._find_leaf(low)

    while node:
        for k, v in zip(node.keys, node.children):
            if low <= k <= high:
                result.append((k, v))

            elif k > high:
                return result

        node = node.next

    return result

def display_leaves(self):
    node = self.root

    while not node.leaf:
        node = node.children[0]

    print("Leaf chain (via next pointers):")

    while node:
        print(" ", node.keys, end=" -> " if node.next else " -> None\n")

        node = node.next

if __name__ == "__main__":
    tree = BPlusTree(order=4)

```

```
for k in [10, 20, 5, 6, 12, 30, 7, 17, 25, 3]:  
    tree.insert(k, f"val{k}")  
tree.display_leaves()  
print("Search 12:", tree.search(12))  
print("Range query [10, 25]:", tree.range_query(10, 25))
```

Sample Output

=== q5_bplustree.py ===

Leaf chain (via next pointers):

[3, 5] -> [6, 7] -> [10, 12, 17] -> [20, 25, 30] -> None

Search 12: val12

Range query [10, 25]: [(10, 'val10'), (12, 'val12'), (17, 'val17'), (20, 'val20'), (25, 'val25')]

Q6. Dense Primary Index on a Sorted File

Bloom's Level: BL4 - Analyze | Marks: 5

Approach

A dense index contains exactly one index entry for every record in the data file -- (key, position) pairs. Search does a binary search directly on the index (which is small enough to often fit in memory) to get the exact data-file position in $O(\log n)$, avoiding any scanning of the data file itself. The index table is rebuilt/kept sorted on every insert and printed to show its dense (1:1) nature.

Code (Python)

"""

Q6: Dense Primary Index on a sorted data file.

A dense index has ONE index entry for EVERY search-key value in the data file, each pointing to the block/offset where that record lives.

"""

```
import bisect
```

```
class DenseIndexedFile:
```

```
    def __init__(self):
```

```
        self.data_file = []      # sorted list of (key, record) tuples
```

```
        self.index = []         # dense index: list of (key, position_in_data_file)
```

```
    def insert(self, key, record):
```

```
        pos = bisect.bisect_left([k for k, _ in self.data_file], key)
```

```
        self.data_file.insert(pos, (key, record))
```

```
        self._rebuild_index()    # every insert keeps the dense index in sync
```

```
    def _rebuild_index(self):
```

```

        # dense index -> exactly one entry per record

        self.index = [(key, pos) for pos, (key, _) in enumerate(self.data_file)]

    def search(self, key):

        keys = [k for k, _ in self.index]

        pos = bisect.bisect_left(keys, key)

        if pos < len(keys) and keys[pos] == key:

            data_pos = self.index[pos][1]

            return self.data_file[data_pos]

        return None

    def display_index(self):

        print("Dense Index Table:")

        print(f"{'Key':<10}{'Data-file position':<20}")

        for key, pos in self.index:

            print(f"{key:<10}{pos:<20}")

if __name__ == "__main__":

    dif = DenseIndexedFile()

    for k, name in [(30, "Bob"), (10, "Alice"), (50, "Eve"), (20, "Carol"), (40,
"Dan")]:

        dif.insert(k, name)

    dif.display_index()

    print("\nSearch key 40:", dif.search(40))

    print("Search key 99:", dif.search(99))

```

Sample Output

=== q6_dense_index.py ===

Dense Index Table:

Key	Data-file position
10	0
20	1
30	2
40	3

50 4

Search key 40: (40, 'Dan')

Search key 99: None

Q7. Extendible Hashing with Directory Doubling (8 records)

Bloom's Level: BL3 - Apply | Marks: 5

Approach

Each directory slot points to a bucket; a bucket's 'local depth' records how many low-order hash bits distinguish it. Global depth controls the directory size ($2^{\text{global_depth}}$). When a bucket overflows (capacity fixed at 2 records here, deliberately small so overflow is guaranteed within 8 insertions) and its local depth equals the global depth, the WHOLE directory is doubled and the global depth incremented before the bucket is split -- this is the directory-doubling event the question asks to demonstrate. If local depth is already less than global depth, only the bucket splits and the directory need not grow.

Code (Python)

```
"""
```

Q7: Extendible Hashing.

Shows directory doubling when a bucket overflows during insertion of 8 records. Bucket capacity is kept small (2) so a doubling event is guaranteed to be visible with only 8 insertions.

```
"""
```

```
BUCKET_CAPACITY = 2
```

```
class Bucket:
```

```
    def __init__(self, local_depth):
```

```
        self.local_depth = local_depth
```

```
        self.records = []    # list of (key, value)
```

```
    def is_full(self):
```

```
        return len(self.records) >= BUCKET_CAPACITY
```

```
class ExtendibleHashTable:
```

```
    def __init__(self):
```

```
        self.global_depth = 1
```

```
        b0, b1 = Bucket(1), Bucket(1)
```

```
        self.directory = [b0, b1]    # directory size = 2 ** global_depth
```

```
    @staticmethod
```

```

def _hash(key):
    # last N bits of the binary hash are what matter
    return key

def _suffix(self, key, depth):
    return self._hash(key) & ((1 << depth) - 1)

def insert(self, key, value):
    idx = self._suffix(key, self.global_depth)
    bucket = self.directory[idx]

    if not bucket.is_full():
        bucket.records.append((key, value))
        return

    # bucket is full -> split (and double directory if needed)
    if bucket.local_depth == self.global_depth:
        print(f" Directory doubling triggered by key {key} "
              f"(global depth {self.global_depth} -> {self.global_depth + 1})")
        self.directory = self.directory + self.directory # double directory
        self.global_depth += 1

    self._split_bucket(idx)
    self.insert(key, value) # retry after split

def _split_bucket(self, idx):
    old_bucket = self.directory[idx]
    new_depth = old_bucket.local_depth + 1
    bucket0 = Bucket(new_depth)
    bucket1 = Bucket(new_depth)

    # repoint all directory entries that used to point to old_bucket
    for i in range(len(self.directory)):
        if self.directory[i] is old_bucket:

```

```

        suffix_bit = i & (1 << (new_depth - 1))

        self.directory[i] = bucket1 if suffix_bit else bucket0

# redistribute old records into the two new buckets
for key, value in old_bucket.records:

    target_idx = self._suffix(key, self.global_depth)

    self.directory[target_idx].records.append((key, value))

print(f" Split bucket (old local depth {old_bucket.local_depth}) "
      f"into two buckets with local depth {new_depth}")

def display(self):

    print(f"\nGlobal depth = {self.global_depth}, directory size =
{len(self.directory)}")

    shown = {}

    for i, b in enumerate(self.directory):

        shown.setdefault(id(b), []).append(i)

    for bucket_id, slots in shown.items():

        bucket = self.directory[slots[0]]

        print(f" Directory slots {slots} -> bucket(local
depth={bucket.local_depth}) "
              f"records={bucket.records}")

if __name__ == "__main__":

    eht = ExtendibleHashTable()

    keys = [4, 12, 8, 5, 13, 9, 20, 3]    # 8 records to insert

    for k in keys:

        print(f"Inserting key {k} (binary: {bin(k)})")

        eht.insert(k, f"rec{k}")

    eht.display()

```

Sample Output

=== q7_extendible_hashing.py ===

Inserting key 4 (binary: 0b100)

Inserting key 12 (binary: 0b1100)

Inserting key 8 (binary: 0b1000)

Directory doubling triggered by key 8 (global depth 1 -> 2)

Split bucket (old local depth 1) into two buckets with local depth 2

Directory doubling triggered by key 8 (global depth 2 -> 3)

Split bucket (old local depth 2) into two buckets with local depth 3

Inserting key 5 (binary: 0b101)

Inserting key 13 (binary: 0b1101)

Inserting key 9 (binary: 0b1001)

Split bucket (old local depth 1) into two buckets with local depth 2

Split bucket (old local depth 2) into two buckets with local depth 3

Inserting key 20 (binary: 0b10100)

Directory doubling triggered by key 20 (global depth 3 -> 4)

Split bucket (old local depth 3) into two buckets with local depth 4

Inserting key 3 (binary: 0b11)

Global depth = 4, directory size = 16

Directory slots [0, 8] -> bucket(local depth=3) records=[(8, 'rec8')]

Directory slots [1, 9] -> bucket(local depth=3) records=[(9, 'rec9')]

Directory slots [2, 6, 10, 14] -> bucket(local depth=2) records=[]

Directory slots [3, 7, 11, 15] -> bucket(local depth=2) records=[(3, 'rec3')]

Directory slots [4] -> bucket(local depth=4) records=[(4, 'rec4'), (20, 'rec20')]

Directory slots [5, 13] -> bucket(local depth=3) records=[(5, 'rec5'), (13, 'rec13')]

Directory slots [12] -> bucket(local depth=4) records=[(12, 'rec12')]

Q8. Secondary Storage Block Access: Sequential vs Indexed I/O

Bloom's Level: BL4 - Analyze | Marks: 5

Approach

With a blocking factor of 10 records/block and 10,000 total records (1,000 blocks), the simulator computes worst-case I/O counts: sequential retrieval must read every block up to and including the one holding the target record ($\text{ceil}(\text{position} / \text{blocking_factor})$ reads); indexed retrieval reads only the small number of index-level blocks plus exactly one data block; and, for comparison, binary search on a sorted file costs about $\text{ceil}(\log_2(\text{num_blocks}))$ reads. The numbers make the huge I/O saving of indexing over sequential scanning concrete.

Code (Python)

```
"""
```

Q8: Simulate secondary storage block access and calculate the number of I/O operations (block reads) for:

(a) Sequential retrieval - scan every block until the record is found

(b) Indexed retrieval - 1 I/O for the index block(s) + 1 I/O for the data block

```
"""
```

```
BLOCKING_FACTOR = 10    # records per block (typical DB textbook value)
```

```
class StorageSimulator:
```

```
    def __init__(self, num_records, blocking_factor=BLOCKING_FACTOR):
```

```
        self.num_records = num_records
```

```
        self.blocking_factor = blocking_factor
```

```
        self.num_blocks = -(-num_records // blocking_factor) # ceil division
```

```
    def sequential_search_io(self, target_position):
```

```
        """Records stored unordered/sequential file: worst case scans up to
        the block containing target_position (1-indexed record position)."""
```

```
        block_containing_target = -(-target_position // self.blocking_factor)
```

```
        print(f"Sequential retrieval: must read blocks 1..{block_containing_target}")
```

```
        return block_containing_target
```

```
    def indexed_search_io(self, index_levels=1):
```

```
        """With a primary/dense index: read index block(s) ( $\log_f(N)$  for a
```

```

        multi-level / B-tree style index) then ONE data block."""
        io_for_index = index_levels
        io_for_data = 1
        total = io_for_index + io_for_data
        print(f"Indexed retrieval: {io_for_index} index-block read(s) "
              f"+ {io_for_data} data-block read = {total} I/O")
        return total

def binary_search_io(self):
    """Binary search directly on a sorted data file (no separate index)."""
    import math

    io = math.ceil(math.log2(self.num_blocks + 1))

    print(f"Binary search on sorted file: ceil(log2({self.num_blocks})) = {io} "
          I/O")

    return io

if __name__ == "__main__":
    N = 10000    # total records

    sim = StorageSimulator(N)

    print(f"Total records = {N}, blocking factor = {BLOCKING_FACTOR}, "
          f"total blocks = {sim.num_blocks}\n")

    target = 8734    # looking for the record at logical position 8734

    seq_io = sim.sequential_search_io(target)

    idx_io = sim.indexed_search_io(index_levels=2)    # e.g. 2-level sparse index

    bin_io = sim.binary_search_io()

    print("\nSummary:")

    print(f"  Sequential scan worst case : {seq_io} I/Os")

    print(f"  Indexed retrieval           : {idx_io} I/Os")

    print(f"  Binary search (sorted file): {bin_io} I/Os")

    print(f"  Speed-up (sequential/index): {seq_io / idx_io:.1f}x")

```

Sample Output

=== q8_block_access.py ===

Total records = 10000, blocking factor = 10, total blocks = 1000

Sequential retrieval: must read blocks 1..874

Indexed retrieval: 2 index-block read(s) + 1 data-block read = 3 I/O

Binary search on sorted file: $\text{ceil}(\log_2(1000)) = 10$ I/O

Summary:

Sequential scan worst case : 874 I/Os

Indexed retrieval : 3 I/Os

Binary search (sorted file): 10 I/Os

Speed-up (sequential/index): 291.3x

Q9. Sparse Index on a Sorted File, Compared to Dense Index (Timing)

Bloom's Level: BL4 - Analyze | Marks: 5

Approach

A sparse index stores only ONE entry per BLOCK (here, one entry per 20 records) rather than one per record. To search, we binary-search the sparse index to find the block that could contain the key, then linearly scan just that block. This uses far less index memory than a dense index but requires a short in-block scan per lookup. Python's `time.perf_counter` is used to time 2,000 random lookups against both structures on 20,000 records, quantifying the small extra cost of the sparse index's in-block scan versus its large memory saving.

Code (Python)

```
"""
```

Q9: Sparse Index on a sorted file, compared against a Dense Index

using timing analysis.

- Dense index: one entry per record.

- Sparse index: one entry per BLOCK (assumes data file is sorted); to find a record you binary-search the sparse index for the block, then scan that block sequentially.

```
"""
```

```
import bisect
```

```
import random
```

```
import time
```

```
BLOCK_SIZE = 20    # records per block, used for the sparse index
```

```

class DenseIndex:

    def __init__(self, sorted_data):

        self.data = sorted_data                # list of (key, value), sorted

        self.index = [k for k, _ in sorted_data]    # one entry per record

    def search(self, key):

        pos = bisect.bisect_left(self.index, key)

        if pos < len(self.index) and self.index[pos] == key:

            return self.data[pos]

        return None

class SparseIndex:

    def __init__(self, sorted_data, block_size=BLOCK_SIZE):

        self.data = sorted_data

        self.block_size = block_size

        # one index entry per block: (first_key_in_block, block_start_position)

        self.index = [(sorted_data[i][0], i) for i in range(0, len(sorted_data),
block_size)]

        self.index_keys = [k for k, _ in self.index]

    def search(self, key):

        pos = bisect.bisect_right(self.index_keys, key) - 1

        if pos < 0:

            return None

        block_start = self.index[pos][1]

        block_end = min(block_start + self.block_size, len(self.data))

        for i in range(block_start, block_end):    # scan within the block

            if self.data[i][0] == key:

                return self.data[i]

        return None

def time_lookups(index_obj, keys_to_find, label):

    start = time.perf_counter()

    for k in keys_to_find:

        index_obj.search(k)

```



```
elapsed = time.perf_counter() - start

print(f"{label}: {len(keys_to_find)} lookups in {elapsed * 1000:.3f} ms")

return elapsed

if __name__ == "__main__":

    N = 20000

    data = sorted([(k, f"val{k}") for k in range(N)], key=lambda r: r[0])

    dense = DenseIndex(data)
```

```

sparse = SparseIndex(data, block_size=BLOCK_SIZE)

print(f"Dense index entries : {len(dense.index)}")

print(f"Sparse index entries: {len(sparse.index)}  (1 per {BLOCK_SIZE}
records)\n")

random.seed(42)

lookup_keys = [random.randint(0, N - 1) for _ in range(2000)]

dense_time = time_lookups(dense, lookup_keys, "Dense index lookup")

sparse_time = time_lookups(sparse, lookup_keys, "Sparse index lookup")

print(f"\nDense index uses more memory but is marginally faster per lookup;")

print(f"Sparse index uses ~{BLOCK_SIZE}x less index space at the cost of a
small")

print(f"in-block scan. Ratio sparse/dense time: {sparse_time / dense_time:.2f}x")

```

Sample Output

=== q9_sparse_index.py ===

Dense index entries : 20000

Sparse index entries: 1000 (1 per 20 records)

Dense index lookup: 2000 lookups in 0.887 ms

Sparse index lookup: 2000 lookups in 2.212 ms

Dense index uses more memory but is marginally faster per lookup;

Sparse index uses ~20x less index space at the cost of a small

in-block scan. Ratio sparse/dense time: 2.49x

Q10. B+ Tree Construction, Traversal, and Range Query [15, 45]

Bloom's Level: BL4 - Analyze | Marks: 5

Approach

The same leaf-linked B+ Tree structure from Q5 is built from 14 keys. `level_order_traversal` prints the tree breadth-first, showing the routing keys at each internal level and the actual data-bearing keys at the leaf level. The range query descends once to the leaf containing key 15, then follows leaf 'next' pointers, collecting every key k with $15 \leq k \leq 45$ and stopping as soon as a leaf key exceeds 45 -- demonstrating why B+ Trees are the standard structure for efficient range queries in real database indexes.

Code (Python)

```
"""
Q10: Construct and traverse a B+ Tree, then execute a range query
for keys between 15 and 45, showing all matching records.

(Reuses the same B+ Tree engine as Q5.)
"""

ORDER = 4

class Node:
    def __init__(self, leaf=True):
        self.leaf = leaf
        self.keys = []
        self.children = []
        self.next = None

class BPlusTree:
    def __init__(self, order=ORDER):
        self.max_keys = order - 1
        self.root = Node(leaf=True)

    def _find_leaf(self, key):
        node = self.root
        while not node.leaf:
            i = 0
```

```

        while i < len(node.keys) and key >= node.keys[i]:
            i += 1

        node = node.children[i]

    return node

def insert(self, key, value):
    leaf = self._find_leaf(key)

    i = 0

    while i < len(leaf.keys) and leaf.keys[i] < key:
        i += 1

    leaf.keys.insert(i, key)

    leaf.children.insert(i, value)

    if len(leaf.keys) > self.max_keys:
        self._split_leaf(leaf)

def _split_leaf(self, leaf):
    mid = len(leaf.keys) // 2

    new_leaf = Node(leaf=False)

    new_leaf.keys = leaf.keys[mid:]

    new_leaf.children = leaf.children[mid:]

    leaf.keys = leaf.keys[:mid]

    leaf.children = leaf.children[:mid]

    new_leaf.next = leaf.next

    leaf.next = new_leaf

    self._insert_in_parent(leaf, new_leaf.keys[0], new_leaf)

def _insert_in_parent(self, left, key, right):
    if left is self.root:
        new_root = Node(leaf=False)

        new_root.keys = [key]

        new_root.children = [left, right]

        self.root = new_root

        return

    parent = self._find_parent(self.root, left)

```

```
i = parent.children.index(left)
```

```
parent.keys.insert(i, key)
```

```

        parent.children.insert(i + 1, right)

    if len(parent.keys) > self.max_keys:
        self._split_internal(parent)

def _split_internal(self, node):
    mid = len(node.keys) // 2
    up_key = node.keys[mid]
    new_node = Node(leaf=False)
    new_node.keys = node.keys[mid + 1:]
    new_node.children = node.children[mid + 1:]
    node.keys = node.keys[:mid]
    node.children = node.children[:mid + 1]
    self._insert_in_parent(node, up_key, new_node)

def _find_parent(self, current, child):
    for c in current.children:
        if isinstance(c, Node) and not c.leaf:
            if child in c.children:
                return c
            found = self._find_parent(c, child)
            if found:
                return found
    if child in current.children:
        return current
    return None

def level_order_traversal(self):
    print("Level-order traversal:")
    level = [self.root]
    depth = 0
    while level:
        row = " ".join(str(n.keys) for n in level)
        print(f"Level {depth}: {row}")
        next_level = []

```

```

        for n in level:
            if not n.leaf:
                next_level.extend(n.children)
            level = next_level
        depth += 1

def range_query(self, low, high):
    node = self._find_leaf(low)
    result = []
    while node:
        for k, v in zip(node.keys, node.children):
            if low <= k <= high:
                result.append((k, v))
            elif k > high:
                return result
        node = node.next
    return result

if __name__ == "__main__":
    tree = BPlusTree(order=4)
    keys = [10, 20, 5, 6, 12, 30, 7, 17, 25, 3, 45, 38, 15, 22]
    for k in keys:
        tree.insert(k, f"record_{k}")
    tree.level_order_traversal()
    print("\nRange query for keys between 15 and 45:")
    matches = tree.range_query(15, 45)
    for k, v in matches:
        print(f"  key={k} -> {v}")

```

Sample Output

=== q10_bplustree_range.py ===

Level-order traversal:

Level 0: [20]

Level 1: [6, 10, 15] [30]

Level 2: [3, 5] [6, 7] [10, 12] [15, 17] [20, 22, 25] [30, 38, 45]

Range query for keys between 15 and 45:

key=15 -> record_15

key=17 -> record_17

key=20 -> record_20

key=22 -> record_22

key=25 -> record_25

key=30 -> record_30

key=38 -> record_38

key=45 -> record_45